

Notas del Docente y Guía de Diagnóstico

Cinemática Inversa (W07-S02)[cite: 10]

Matriz de Diagnóstico y Mitigación Rápida[cite: 10]

Durante el trabajo de los estudiantes, preste atención a las siguientes observaciones para intervenir oportunamente:

Observación	Posible Concepto Erróneo	Intervención Inmediata
Afirma que la IK es T^{-1} .	Inversión de transformación confundida con IK.	Pregunte qué variables contiene físicamente la matriz T^{-1} .
Acepta solo una solución 2R.	Multiplicidad no interiorizada.	Pida que esboce las configuraciones de codo-arriba y codo-abajo.
Trata el signo $\pm$ como arbitrario.	No hay significado de rama geométrica.	Mapee cada signo a una postura física del robot.
Llama "caso único" a un target en el límite.	Conexión con singularidad ausente.	Compare un objetivo interior versus un brazo estirado.
Usa $\tan^{-1}(y/x)$ en todas partes.	Se pierde la información del cuadrante.	Compare analíticamente QI y QIII con coordenadas negativas.
Centro de muñeca = origen de herramienta siempre.	Confusión de offset/marco.	Dibuje la brida y el vector d_6 de retroceso.
No puede derivar $R_3^6 = (R_0^3)^T R_d$.	Composición de rotaciones débil.	Inicie desde la ecuación base $R_0^6 = R_0^3 R_3^6$.
El marco D-H debe ser igual al marco URDF.	Representación / Geometría confundidas.	Recuerde que el vector físico es invariante, las coordenadas no.
Trata axis de URDF como global.	Semántica del marco de junta débil.	Pida que identifique el marco local de la junta primero.
Busca las ocho ramas exactas de inmediato.	Conflación de ruta analítica con la derivación final completa.	Restablezca el límite de la sesión: hoy es solo la arquitectura general.
Verifica solo la posición para la pose completa.	Orientación omitida en validación.	Requiera comparación tanto de matriz R como vector p .

Código de Verificación Opcional (Python 2R IK)[cite: 10]

El siguiente código implementa analíticamente la cinemática inversa de 2R considerando la multiplicidad, ideal para verificar los cálculos manuales en el aula.

```
import numpy as np

a1 = 1.0
a2 = 1.0
x, y = 1.0, 1.0

c2 = (x*x + y*y - a1*a1 - a2*a2) / (2*a1*a2)

for sign in (+1, -1):
    s2 = sign * np.sqrt(max(0.0, 1 - c2*c2))
```

```
q2 = np.arctan2(s2, c2)
q1 = np.arctan2(y, x) - np.arctan2(a2*s2, a1 + a2*c2)

x_fk = a1*np.cos(q1) + a2*np.cos(q1 + q2)
y_fk = a1*np.sin(q1) + a2*np.sin(q1 + q2)

print(np.round([q1, q2], 6), np.round([x_fk, y_fk], 6))

# Salidas esperadas:
# [0.000000, 1.570796] -> [1.0, 1.0]
# [1.570796, -1.570796] -> [1.0, 1.0]
```